

Aula 08: Pilhas

Prof. Dr. Lucas Pascotti Valem

Universidade de São Paulo (USP)

ICMC - Instituto de Ciências Matemáticas e de Computação



O que é?

Pilha (*Stack*)

Uma **pilha** é uma estrutura de dados linear do tipo **LIFO** (*Last-In, First-Out*), ou seja, o **último elemento inserido é o primeiro a ser removido**. Imagine uma pilha de pratos: você sempre coloca um prato no topo e também remove o do topo.



Operações Fundamentais

Operações

Operações fundamentais:

1. **push(e)**: insere o elemento *e* no topo da pilha.
1. **pop()**: remove e retorna o elemento do topo da pilha.
1. **top()**: ou **peek()**: retorna (sem remover) o elemento do topo da pilha.
1. **is_empty()**: verifica se a pilha está vazia.
1. **is_full()**: (caso de pilha com tamanho fixo) verifica se a pilha está cheia.
1. **free_stack()**: libera a memória alocada para a pilha.

Qual a saída dessas chamadas?

Push 5

Pop

Push 8

Pop

Push 9

Push 10

Push 50

Push 8

Peek

Pop

Pop

Pop

Qual a saída dessas chamadas?

Push 5

Pop

Push 8

Pop

Push 9

Push 10

Push 50

Push 8

Peek

Pop

Pop

Pop

Saída:

5, 8, 8, 8, 50, 10

Aplicações

Aplicações em Computação

- **Undo/Redo em editores:** cada ação do usuário pode ser empilhada para desfazer (pop) ou refeita (push em outra pilha).
- **Navegação em navegadores:** o histórico de páginas visitadas é gerenciado como uma pilha (voltar e avançar).
- **Compiladores e interpretadores:** usam pilhas para análise sintática, geração de código e controle de escopo de variáveis.
- **Sistema de chamadas de funções (call stack):** armazena informações de funções ativas: parâmetros, variáveis locais e endereço de retorno. Essencial para execução de programas.
- **Gerenciamento de memória e processos:** muitas arquiteturas usam pilha para alocar e desalocar memória temporária durante a execução.
- **Avaliação de expressões:** pilhas são usadas para resolver expressões matemáticas (ex: conversão de infixa para pós-fixa).

... (são muitas!)

TAD Pilha

TAD Pilha (Tipo Abstrato de Dados)

O **TAD Pilha** define a **interface e o comportamento esperado** de uma pilha, independentemente da implementação interna (vetor, lista, etc.). Isso significa que o usuário do TAD pode manipular a estrutura apenas por meio das **operações definidas na interface, sem precisar conhecer os detalhes de como os dados são armazenados.**

TAD Pilha (Tipo Abstrato de Dados)

O **TAD Pilha** define a **interface e o comportamento esperado** de uma pilha, independentemente da implementação interna (vetor, lista, etc.). Isso significa que o usuário do TAD pode manipular a estrutura apenas por meio das **operações definidas na interface, sem precisar conhecer os detalhes de como os dados são armazenados.**

A principal vantagem de utilizar um TAD é abstrair a implementação: **a pilha pode ser implementada de forma estática (com vetor de tamanho fixo) ou dinâmica (com lista ou vetor redimensionável)**, mas o código que a utiliza permanece o mesmo, desde que a interface definida seja respeitada.

Interface de uma Pilha

Interface típica (em C):

Stack.h

```
1  #ifndef STACK_H
2  #define STACK_H
3
4  typedef struct stack Stack;
5
6  Stack* create_stack(void);
7  int is_empty(const Stack *s);
8  int is_full(const Stack *s);
9  int push(Stack *s, int value);
10 int pop(Stack *s, int *value);
11 int peek(const Stack *s, int *value);
12 void free_stack(Stack **s);
13
14 #endif
```

Interface de uma Pilha

Interface típica (em C):

Definimos o tipo Stack como **tipo opaco**.

- Em C, “tipo opaco” significa declarar o tipo como incompleto no .h e definir a struct só no .c.
- Quem usa a pilha não vê os campos, não pode fazer `sizeof(Stack)` e não pode declarar `Stack s`; apenas `Stack*`.
- Criação e destruição ficam na API. Assim, você pode trocar vetor por lista no futuro sem mudar o código do usuário.

Stack.h

```
1  #ifndef STACK_H
2  #define STACK_H
3
4  typedef struct stack Stack;
5
6  Stack* create_stack(void);
7  int is_empty(const Stack *s);
8  int is_full(const Stack *s);
9  int push(Stack *s, int value);
10 int pop(Stack *s, int *value);
11 int peek(const Stack *s, int *value);
12 void free_stack(Stack **s);
13
14 #endif
```

Interface de uma Pilha

Interface típica (em C):

Note que as **funções** são as apresentadas nos slides anteriores. Vamos implementá-las juntos!

Stack.h

```
1  #ifndef STACK_H
2  #define STACK_H
3
4  typedef struct stack Stack;
5
6  Stack* create_stack(void);
7  int is_empty(const Stack *s);
8  int is_full(const Stack *s);
9  int push(Stack *s, int value);
10 int pop(Stack *s, int *value);
11 int peek(const Stack *s, int *value);
12 void free_stack(Stack **s);
13
14 #endif
```

Convenções de Retorno das Funções

Convenções de Retorno:

- **-1** → Erro
 - Ponteiro NULL, posição inválida, pilha cheia/vazia, falha de alocação, etc.
- **1** → Sucesso / Condição verdadeira
 - Operação concluída (inserção, remoção, criação...) ou condição satisfeita.
- **0** → Sucesso / Condição falsa
 - Condição não atendida (pilha não vazia, elemento não encontrado).
- **>1** → Valor informativo
 - Ex.: quantidade de elementos.

Obs.: Negativos indicam erro, positivos indicam sucesso.

Struct da Pilha (*Stack*)

Como estamos usando **tipo opaco**, ela é definida no .c da seguinte forma:

```
#include "Stack.h"

#define MAX 100      // capacidade da pilha

struct stack {
    int dados[MAX]; // vetor que armazena os elementos da pilha
    int qtd;        // número de elementos na pilha
};
```

Struct da Pilha (*Stack*)

Como estamos usando **tipo opaco**, ela é definida no .c da seguinte forma:

```
#include "Stack.h"

#define MAX 100      // capacidade da pilha

struct stack {
    int dados[MAX]; // vetor que armazena os elementos da pilha
    int qtd;        // número de elementos na pilha
};
```

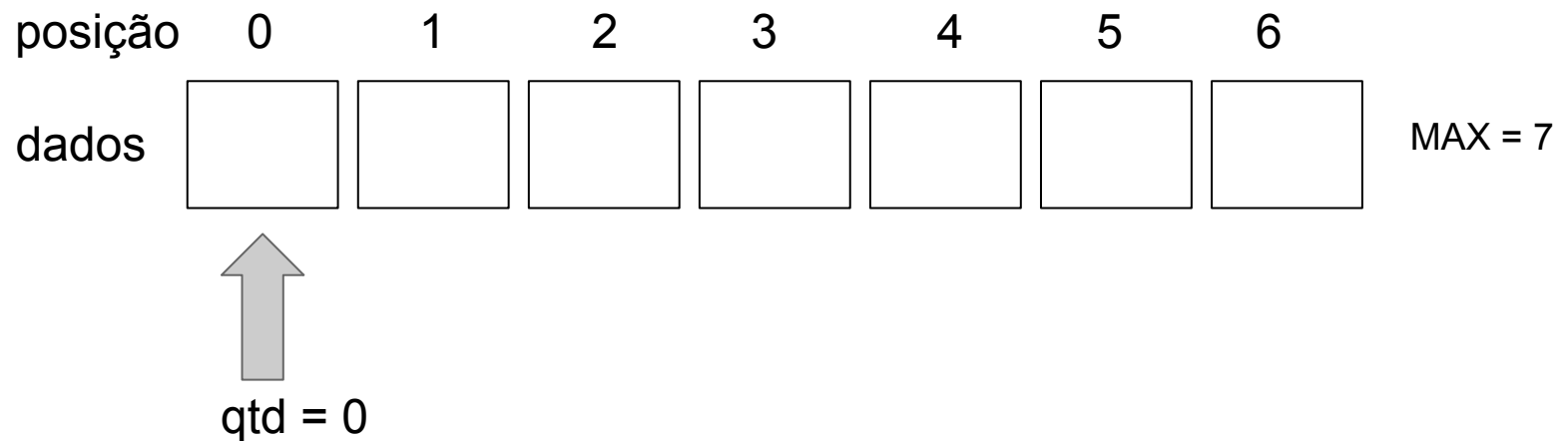
- Nesta implementação, usaremos um vetor de tamanho fixo, estático.
- É muito parecido com uma Lista Sequencial Estática (LSE) que veremos nas próximas aulas.

Visualizações

Função *is_empty*
(verifica pilha vazia)

Está vazia?

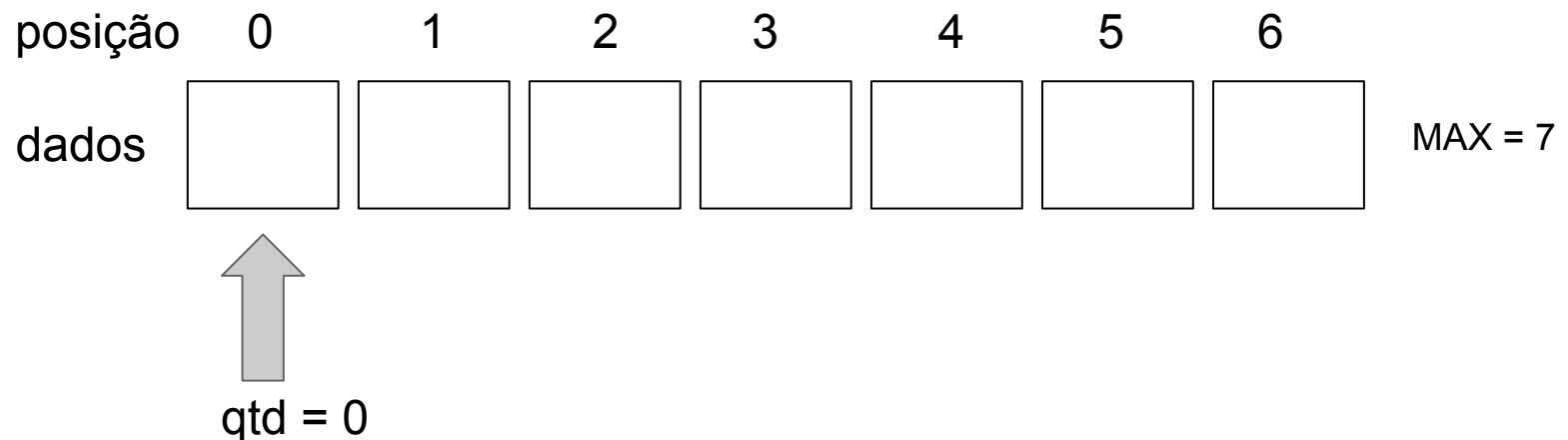
```
int is_empty(Stack *s)
```



Está vazia?

```
int is_empty(Stack *s)
```

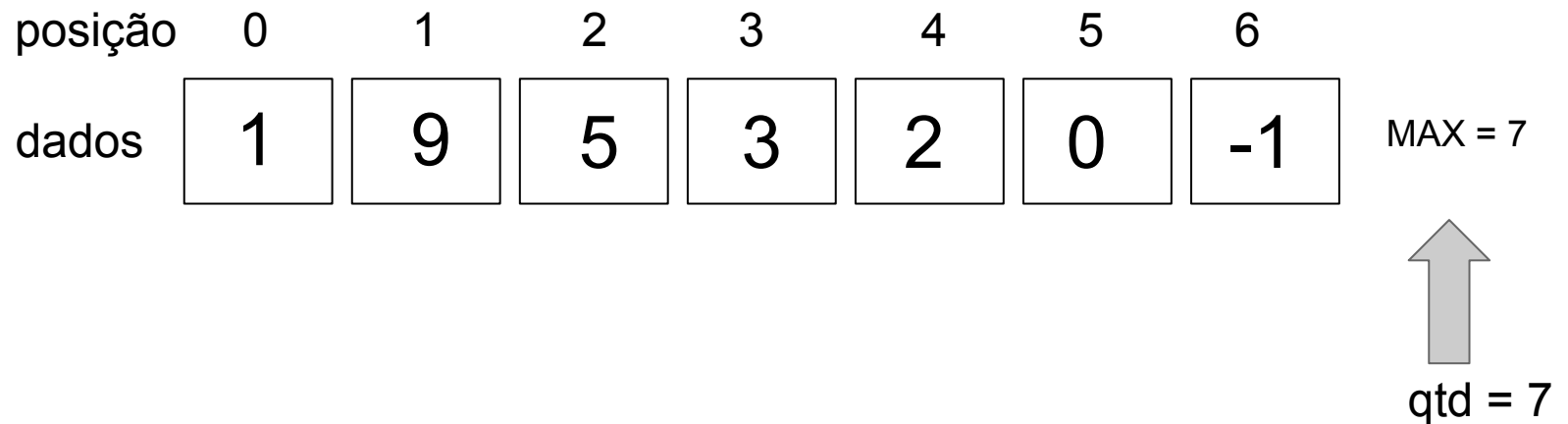
Se chamamos **is_empty(s)**, basta verificar se **s->qtd** é zero para saber se a pilha está **vazia**.



Função *is_full*
(verifica pilha cheia)

Está cheia?

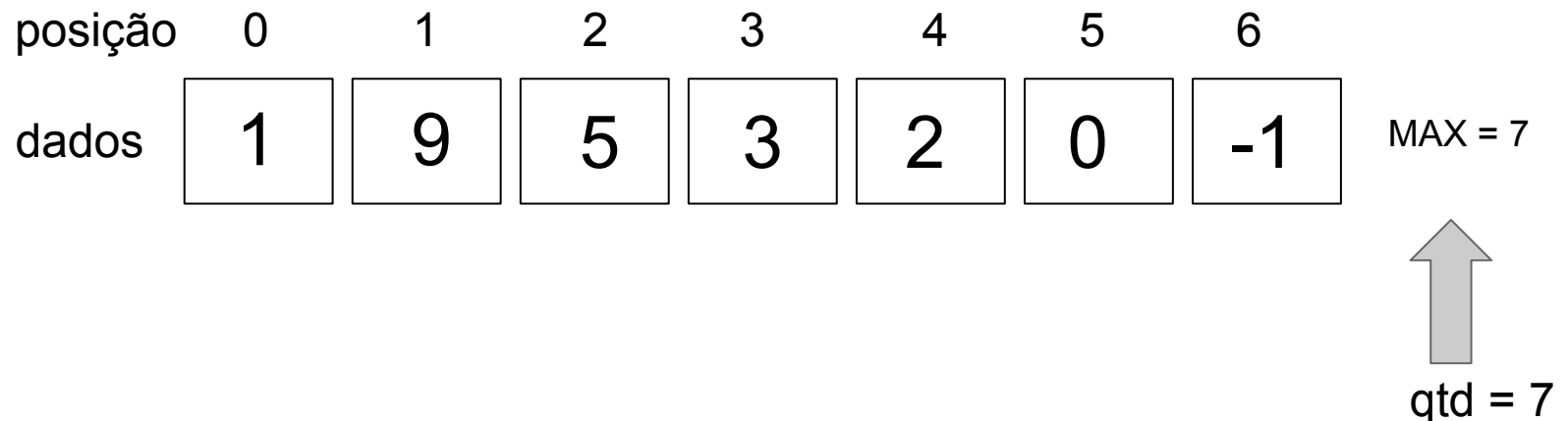
```
int is_full(Stack *s)
```



Está cheia?

```
int is_full(Stack *s)
```

Se chamamos **is_full(s)**, basta verificar se **s->qtd** é igual a **MAX** para saber se a pilha está **cheia**.

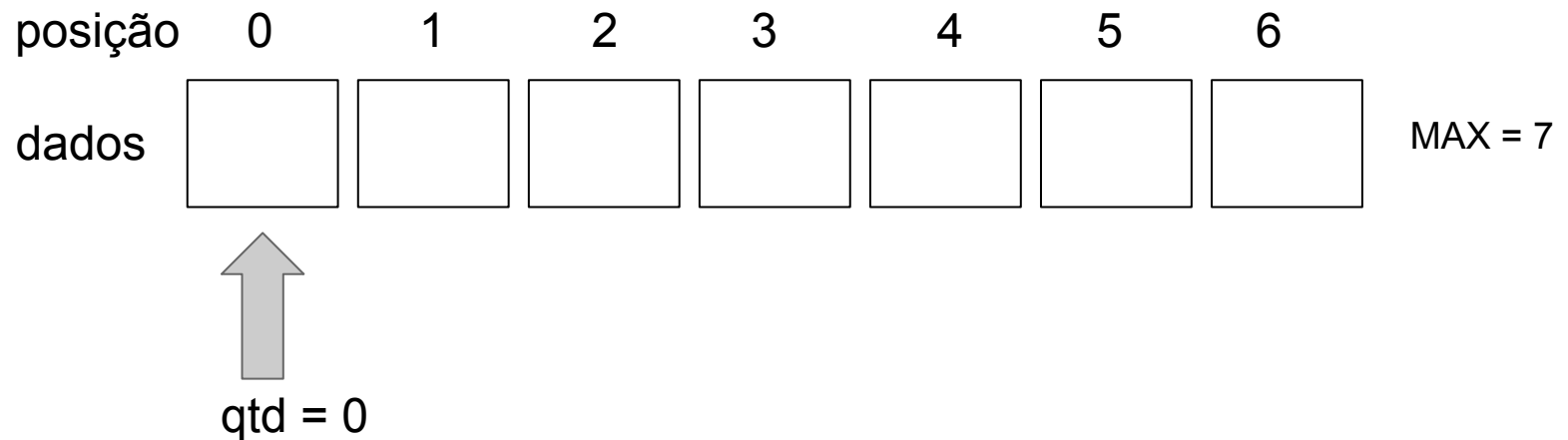


Função *push*
(insere no final)

Inserção no Final

```
int push(Stack *s, int value)
```

Exemplos de inserção no final:

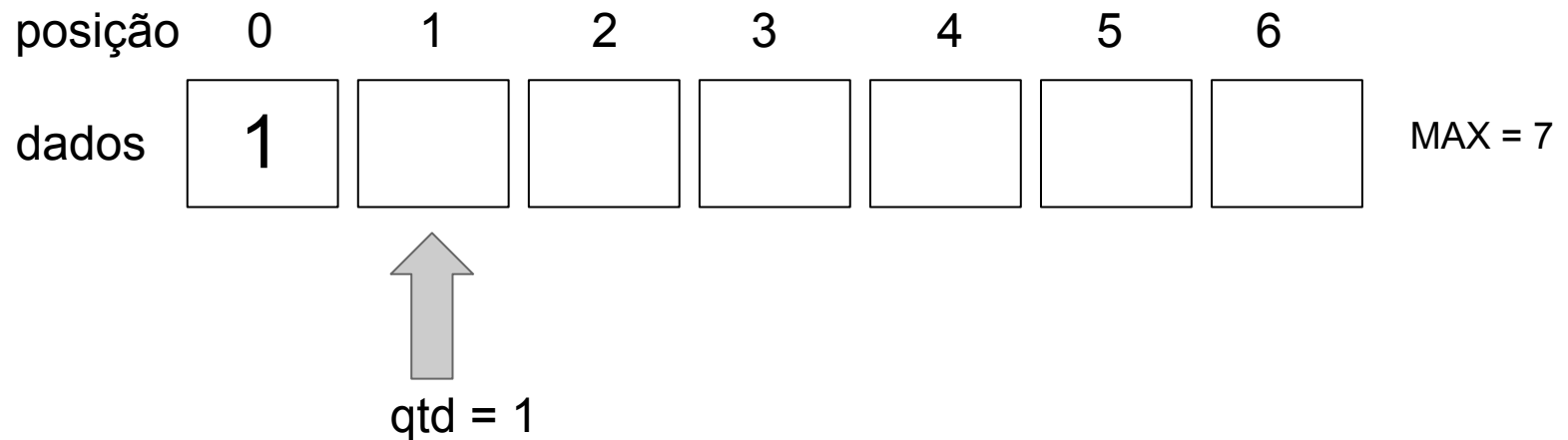


Inserção no Final

```
int push(Stack *s, int value)
```

Exemplos de inserção no final:

push(s, 1)



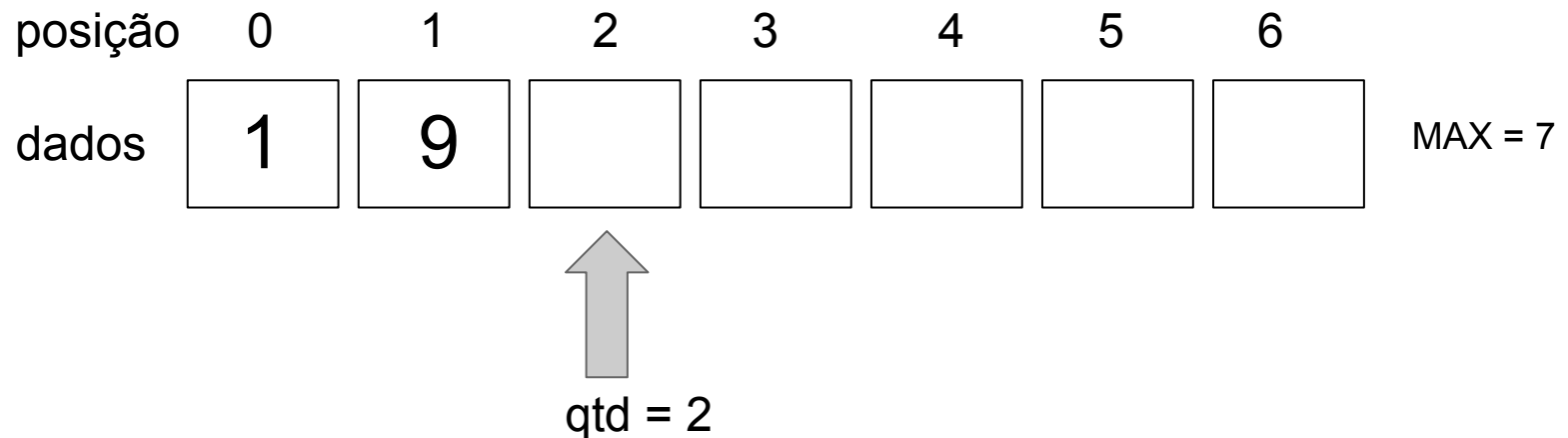
Inserção no Final

```
int push(Stack *s, int value)
```

Exemplos de inserção no final:

push(s, 1)

push(s, 9)



Inserção no Final

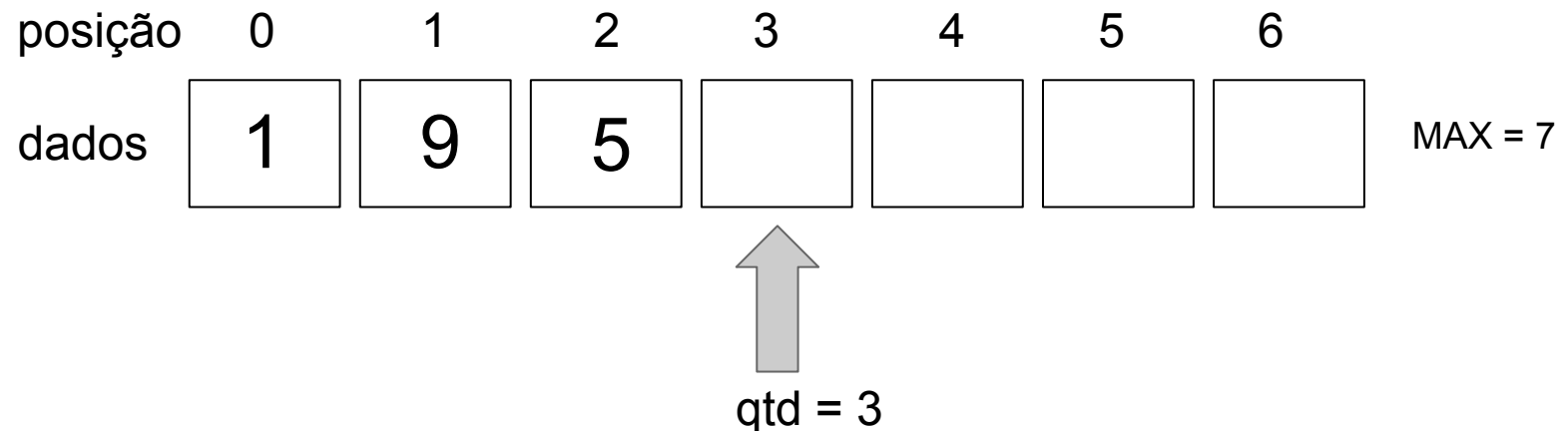
```
int push(Stack *s, int value)
```

Exemplos de inserção no final:

push(s, 1)

push(s, 9)

push(s, 5)



Inserção no Final

```
int push(Stack *s, int value)
```

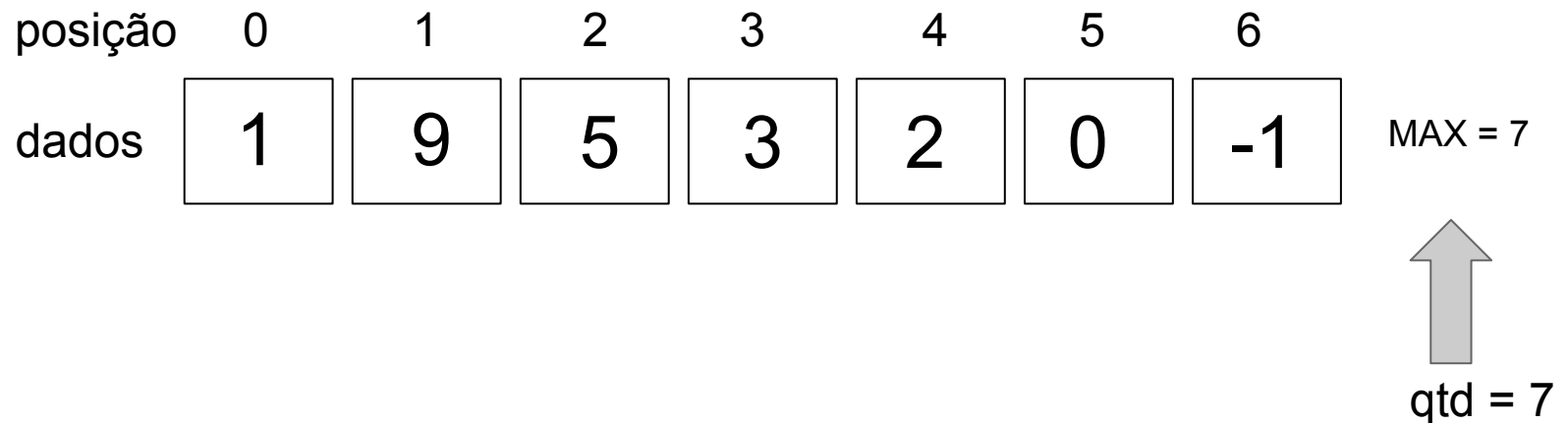
Exemplos de inserção no final:

push(s, 1)

push(s, 9)

push(s, 5)

...



Inserção no Final

```
int push(Stack *s, int value)
```

Exemplos de inserção no final:

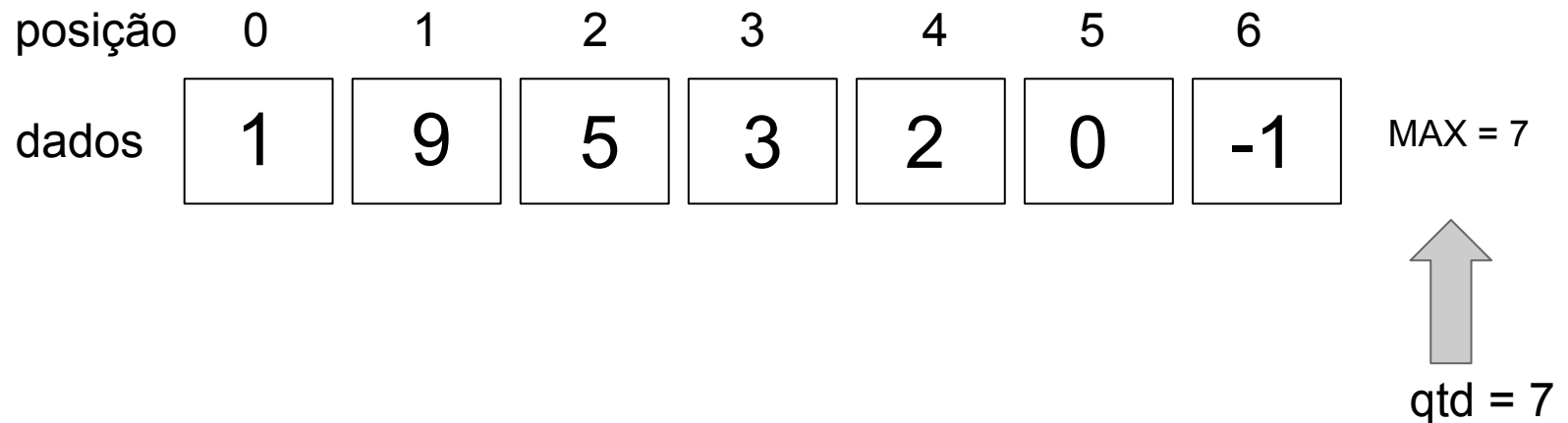
push(s, 1)

push(s, 9)

push(s, 5)

...

push(s, 10)



Inserção no Final

```
int push(Stack *s, int value)
```

Exemplos de inserção no final:

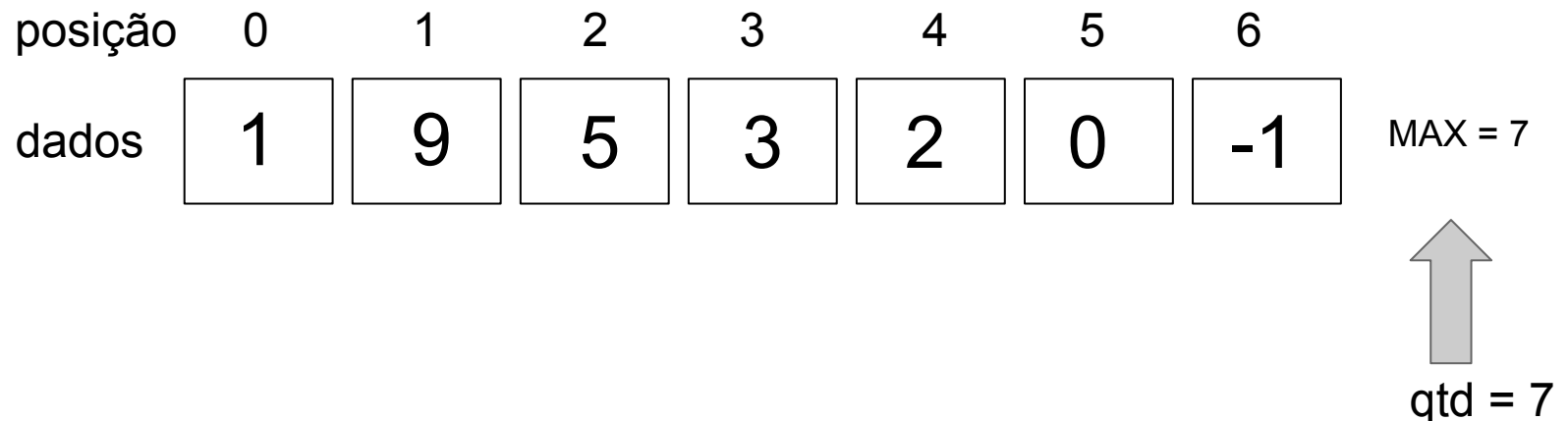
push(s, 1)

push(s, 9)

push(s, 5)

...

push(s, 10) → Retorna 0 (pilha cheia!)

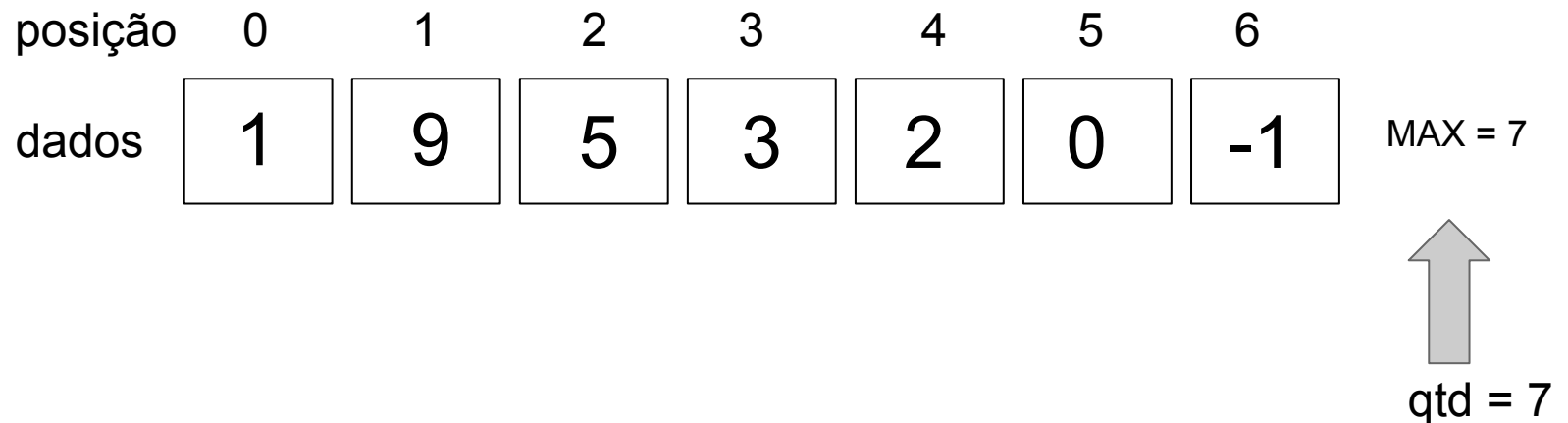


Função *pop*
(remove do final)

Remoção do Final

Int **pop**(Stack *s, int *value)

Exemplos de remoção:

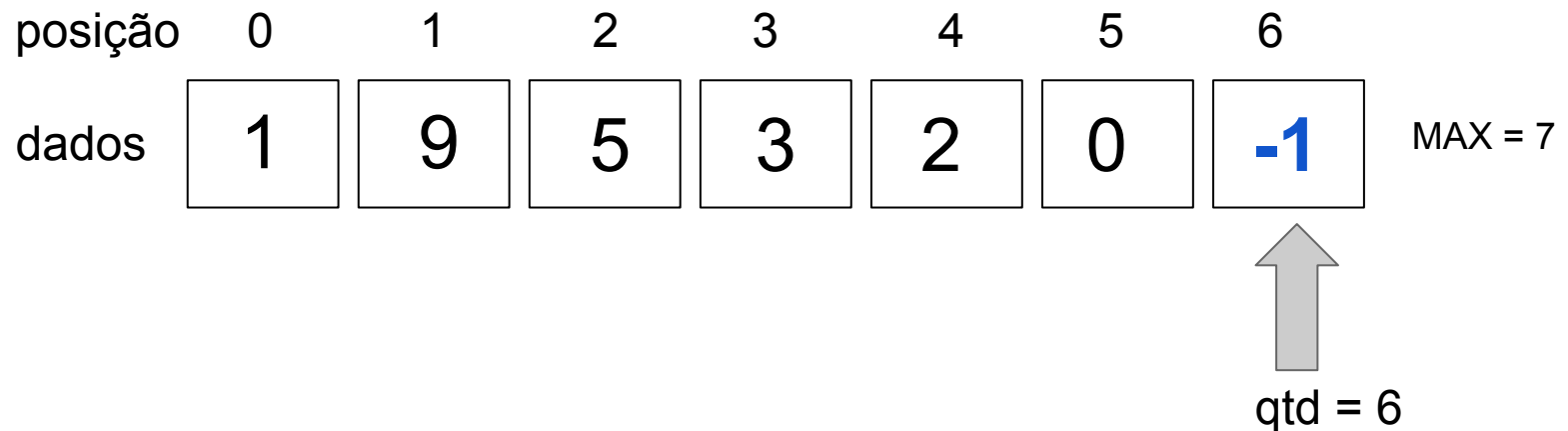


Remoção do Final

Int **pop**(Stack *s, int *value)

Exemplos de remoção:

pop(s, &x) → Retorna 1 e salva -1, que é o elemento removido, em x.



Remoção do Final

Int **pop**(Stack *s, int *value)

Exemplos de remoção:

pop(s, &x) → Retorna 1 e salva -1, que é o elemento removido, em x.

posição	0	1	2	3	4	5	6	
dados	1	9	5	3	2	0	-1	MAX = 7

**Lembre-se: não é necessário apagar o valor removido.
A variável qtd indica até onde a pilha é válida,
valores além disso são ignorados.**

↑
qtd = 6

Remoção do Final

Int **pop**(Stack *s, int *value)

Exemplos de remoção:

pop(s, &x) → Retorna 1 e salva -1, que é o elemento removido, em x.

pop(s, &x) → Retorna 1 e salva 0, que é o elemento removido, em x.

posição	0	1	2	3	4	5	6	
dados	1	9	5	3	2	0	-1	MAX = 7

**Lembre-se: não é necessário apagar o valor removido.
A variável qtd indica até onde a pilha é válida,
valores além disso são ignorados.**

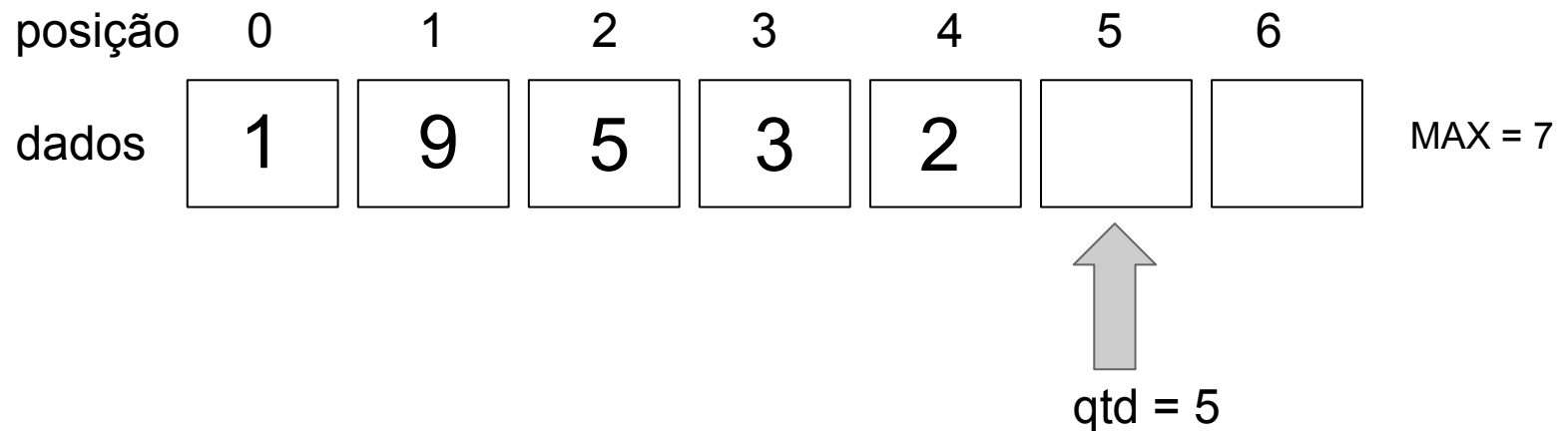
↑
qtd = 5

Função *peek*
(apenas lê o topo)

Lê o topo

Int **peek**(Stack *s, int *value)

Exemplos de remoção:



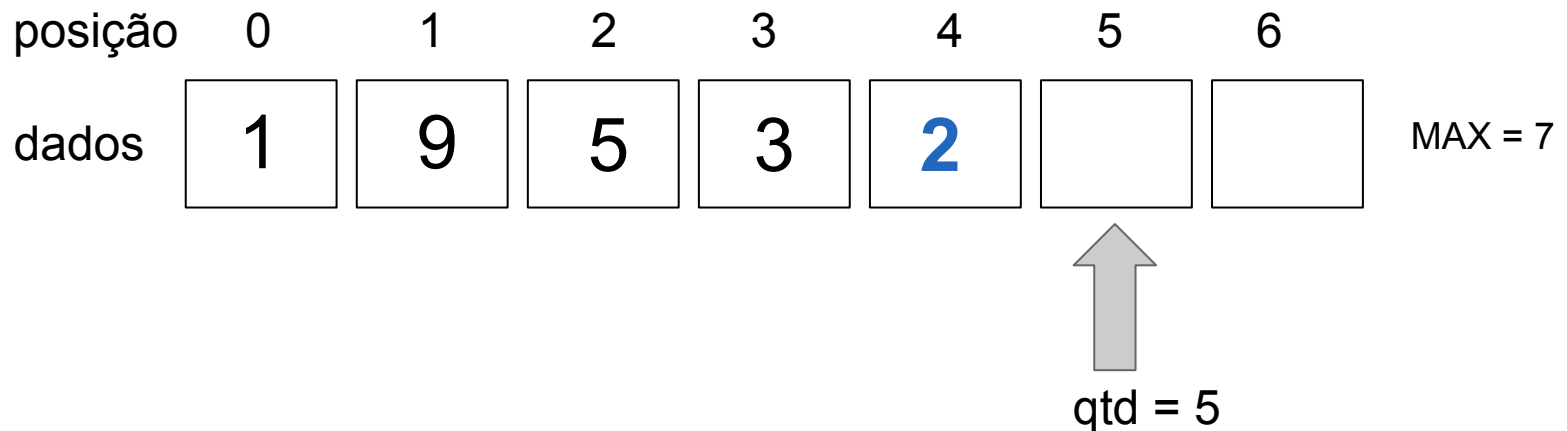
Lê o topo

Int **peek**(Stack *s, int *value)

Exemplos de remoção:

peek(s, &x) → Retorna 1 e salva 2 em x.

**Retorna-se o elemento que está na posição qtd-1: s->dados[s->qtd-1].
Nada é removido e nem inserido. Simples assim!**



Usando o TAD

Usando o TAD no main.c

Agora vamos fazer exercícios criando o nosso main.c que importa o nosso Stack.h

Exemplo de uso do TAD

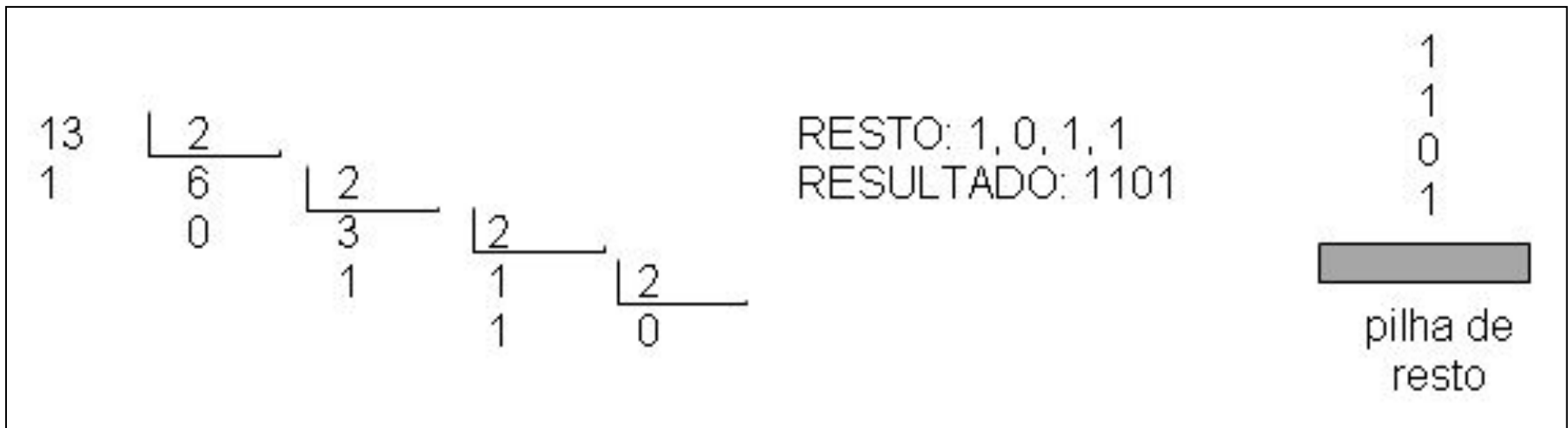


```
C Main.c × +  
  
1  #include <stdio.h>  
2  #include "Stack.h"  
3  
4  int main() {  
5      Stack *s = create_stack();  
6      int x;  
7  
8      push(s, 42);  
9      push(s, 99);  
10  
11     peek(s, &x);  
12     printf("Topo: %d\n", x);  
13  
14     pop(s, &x);  
15     printf("Removido: %d\n", x);  
16  
17     free_stack(&s);  
18     return 0;  
19 }
```

Exercícios

Exercício 1: Decimal para Binário

Fazer algoritmo de conversão decimal para binário usando nosso TAD pilha (incluir o Stack.h).



Estratégia de resolução:

- A cada divisão por 2, empilha o resto.
- Quando acabar a sequência de divisões (quociente=0), desempilha e escreve todos os elementos.

Exercício 2: Calculadora Polonesa

Implemente uma calculadora polonesa usando nossa pilha. Você lê números e operadores em sequência.

- Para cada número, faz *push*.
- Para cada operador, faz *pop* de dois valores, aplica a operação e faz *push* do resultado.
- No fim, sobra um único valor na pilha que é a resposta.

Exemplo: 3 4 + 2 * 7 /

Resultado = 2

Token	Ação	Pilha (base → topo)
3	push 3	[3]
4	push 4	[3, 4]
+	pop 4 e 3, push 3+4=7	[7]
2	push 2	[7, 2]
*	pop 2 e 7, push 7*2=14	[14]
7	push 7	[14, 7]
/	pop 7 e 14, push 14/7=2	[2]

Instruções de Envio

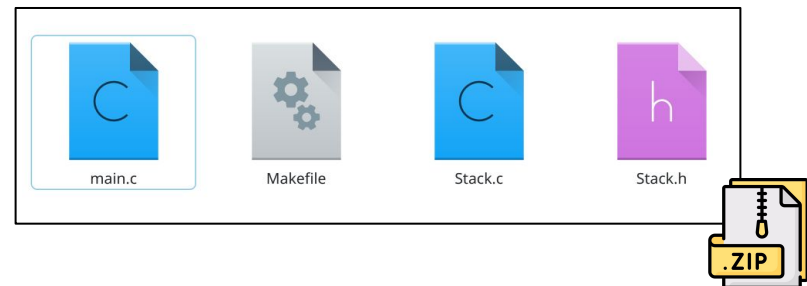
Para enviar no Run.Codes, faça um arquivo **.zip** com:

- **main.c**: Código com implementação solicitada.
- **Stack.h**: Interface do TAD.
- **Stack.c**: Implementação do TAD.
- **Makefile**: Instruções de compilação.

No
Moodle

Arquivos de exemplo no Moodle.

- Lembre-se de escrever corretamente as maiúsculas e minúsculas.
- O correto é sempre "Makefile".



Hora de Praticar!

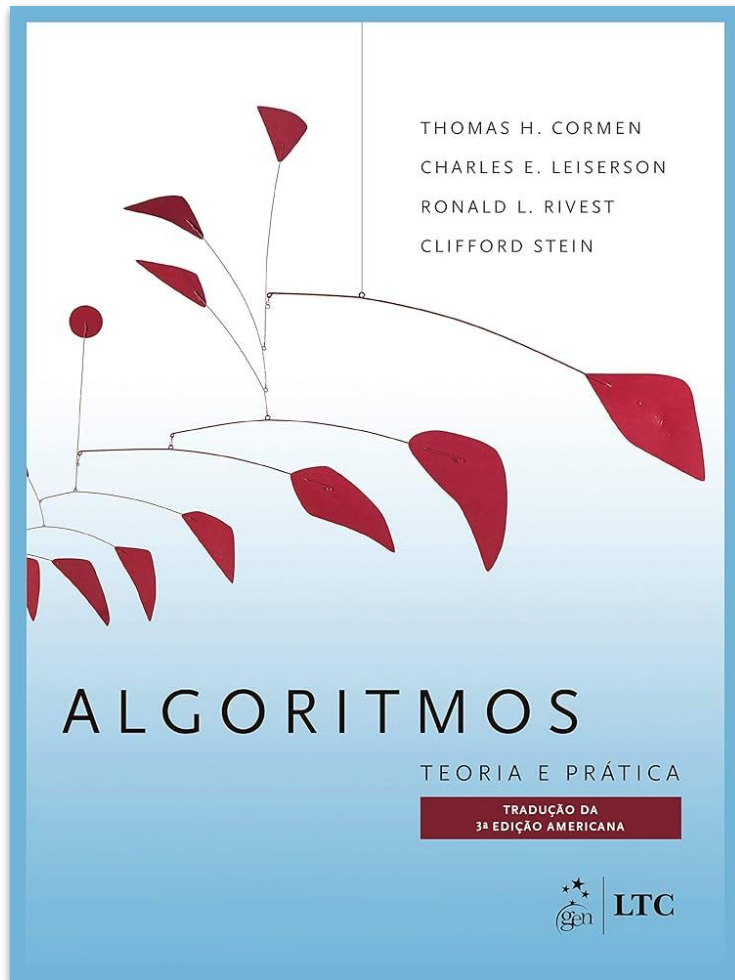
Acessem o:

The logo for run.codes is displayed within a black square. The word "run." is in a bold, white, sans-serif font, and the word "codes" is in a regular weight of the same font, also in white. The text is centered within the square.

run.
codes

Bibliografia

Bibliografia



CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. *Algoritmos: Teoria e Prática*. **Seção 10.1**. Rio de Janeiro: Campus, 2002.